

Thesepapier

Mathis Grün, F12Tb

Zeigen Sie eine einfache Auswertungsanwendung, die Funktionsweise und das Training anhand einer von Ihnen programmierten KI

Herr Fink

18.2.2014

1 EINLEITUNG	3
2 GRUNDLAGEN.....	3
2.1 Definition	3
2.2 Funktionsweise	4
3 MATHEMATISCHE ERLÄUTERUNG	5
3.1 Simulation eines Neurons	5
3.2 Optimierungsalgorithmus	6
3.3 Das Gradientenverfahren.....	7
4 HANDSCHRIFTERKENNUNG: ZAHLEN.....	8
4.1 benötigte Hardware und Software.....	8
4.2 Aufbau	9
5 CODE-IMPLEMENTIERUNG	10
5.1 Grundstruktur des Codes	10
5.2 Trainingsprozess	12
6 EVALUATION.....	13
6.1 Gängige Parameter	13

6.2 Unterschiedliche Parameter.....	15
6.3 Generalization und Orkham's razor.....	17
7 DEMONSTRATION	18
8 FAZIT	18
9 LITERATURVERZEICHNIS.....	18
9.1 Buchquellen	18
9.2 Internetquellen	18
10 ABBILDUNGSVERZEICHNIS.....	20

1 Einleitung

Es wird davon ausgegangen, dass „mindestens 20 Prozent der Schülerinnen und Schüler in Deutschland ChatGPT bereits als Informationsquelle, für Textproduktion und Übersetzung verwenden“¹. Daher wäre es interessant zu wissen, wie eine Künstliche Intelligenz² funktioniert und was innerhalb eines solchen Konstrukts vorgeht.

Das Ziel ist es eine KI-Auswertungsanwendung selbst zu programmieren und anhand dieser das Training erklären. Darüber hinaus soll die Leistung der KI evaluiert werden.

2 Grundlagen

2.1 Definition

Eine KI ist der Versuch, menschliches Denken und Lernen durch Software nachzuahmen. KI ist ein Überbegriff und vereinheitlicht viele Aspekte wie zum Beispiel: *Machine Learning*, *neuronale Netze*, *Representation Learning*, *Deep Learning* und viele Weitere.³ Neuronale Netze versuchen konkret die Prozesse in einem Gehirn nachzuahmen.

¹ Spiegel: Empfehlungen für KI-Nutzung an Schulen.

² Im fortlaufenden Text wird die Abkürzung KI verwendet

³ Vgl: Tiedemann, M.: Wir bringen Licht in die Begriffe rund um das Thema „Künstliche Intelligenz“

2.2 Funktionsweise

Anhand Abbildung 1 wird die neuronale Struktur erklärt, beziehungsweise wie die Neuronen im Gehirn zusammenhängen, und es wird die Frage gestellt, wie diese Prozesse besser visualisiert werden können. Mithilfe von Abbildung 2 wird erklärt, dass Menschen mit ihren Sinnesorganen Reize registrieren. Diese Reize werden verarbeitet und klassifiziert bis das Gehirn erkennt um was es sich handelt. Durch Abbildung 3 wird versucht das Prinzip des Fragens, Nachdenkens und Antwortens auf den Computer zu übertragen.

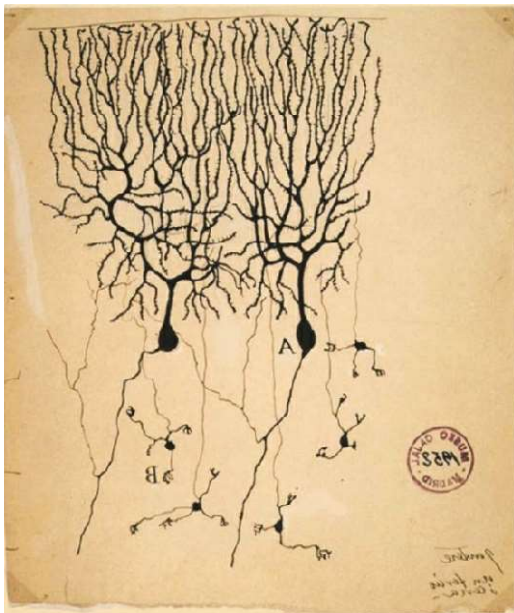


Abbildung 1: Skizze von Neuronen in einem Taubengehirn



Abbildung 2: Schema einer einfachen Vorhersagemaschine



Abbildung 3: Alternative Beschreibung der Vorhersagemaschine

Durch Abbildung 4 wird ein einfaches neuronales Netz mit zwei Schichten und jeweils zwei Neuronen dargestellt. Wichtig: Alle Neuronen der 1. und 2. Schicht, auch *layer* genannt, sind über Gewichte miteinander verbunden. Die erste Schicht wird auch als *input layer* bezeichnet und repräsentiert die Sinnesorgane. Die letzte Schicht wird als *output layer* bezeichnet. Alle Schichten dazwischen sind als *hidden layer* definiert.

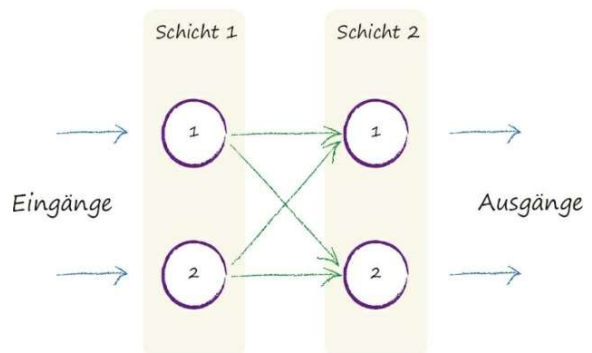


Abbildung 4: Einfaches neuronales Netz

3 Mathematische Erläuterung

3.1 Simulation eines Neurons

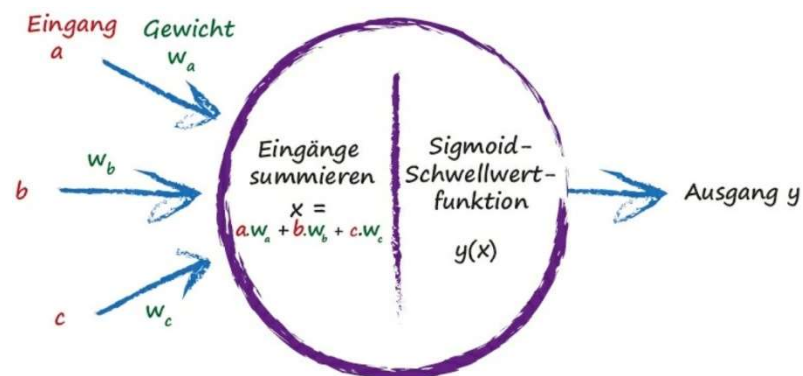
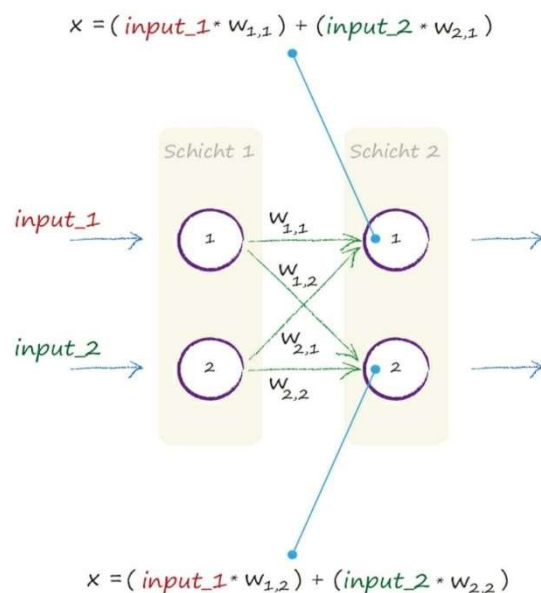


Abbildung 5. Erweitertes Modell eines künstlichen Neurons

Die Abbildung 5 wirft einen genaueren Blick darauf, inwiefern die *inputs* (Eingänge) mit den *weights* (Gewichte) versehen sind. Generell lässt sich für die Gleichung $[x]$ folgende Formel aufstellen:

$$X = a * W_a + b * W_b + c * W_c \dots$$



Diese Formel lässt sich mithilfe von Matrixmultiplikation simpler und effizienter lösen, (s. Abb 6 & 7).

Abbildung 6. Bildung der *inputs* für die zweite Schicht

$$\begin{pmatrix} w_{1,1} & w_{2,1} \\ w_{1,2} & w_{2,2} \end{pmatrix} \begin{pmatrix} \text{input_1} \\ \text{input_2} \end{pmatrix} = \begin{pmatrix} (\text{input_1} * w_{1,1}) + (\text{input_2} * w_{2,1}) \\ (\text{input_1} * w_{1,2}) + (\text{input_2} * w_{2,2}) \end{pmatrix}$$

Abbildung 7. Matrixmultiplikation

Um den Verarbeitungs- oder Berechnungsprozess fertigzustellen, wird eine Schwellenfunktion, bzw. Aktivierungsfunktion benötigt. Bei diesem neuronalen Netz kommt eine *Sigmoid*-Funktion [o] zum Einsatz, welche sich durch folgende Formel ausdrückt:

$$y = \frac{1}{1 + e^{-x}}$$

$$o = \text{sigmoid}(x)$$

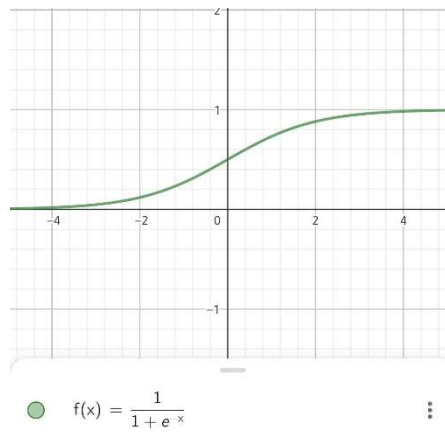


Abbildung 8: Sigmoid Funktion

3.2 Optimierungsalgorithmus

Ein Optimierungsalgorithmus wird benutzt um die Gewichte im neuronalen Netzwerk anzupassen. Der individuelle Fehler [e] eines jeden *outputs* kann mithilfe einer Formel, bestehend aus den festgelegten *target values* und den *outputs*, berechnet werden.⁴ Die nachfolgende Formel dient zur Ermittlung des jeweiligen Fehlers [e]:

$$e = t - o_{\text{output}}$$

Diese Fehler [e] müssen nun in Abhängigkeit ihrer *weights* berechnet werden, wie in Abbildung 9 erkennbar wird.

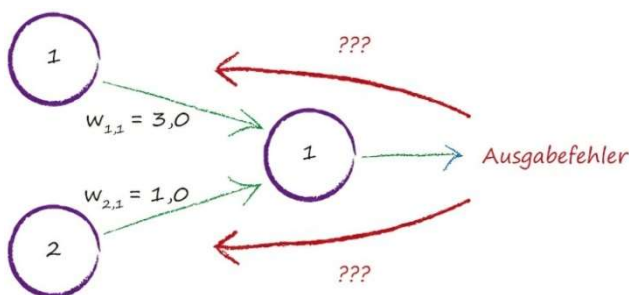


Abbildung 9: Problem bei mehreren inputs

⁴ Vgl: Rashid (2017). Neuronale Netze selbst programmieren, S.16

Auf Abbildung 10 ist die Struktur eines neuronalen Netzes dargestellt, um die *weights* greifbarer zu machen. Dabei werden die drei *layer* in der Abbildung durch die Buchstaben *i*, *j* und *k* repräsentiert. Um die *weights* W_{jk} in Abhängigkeit der Fehler $[e]$ anzupassen, wird das *Gradientenverfahren* genutzt.

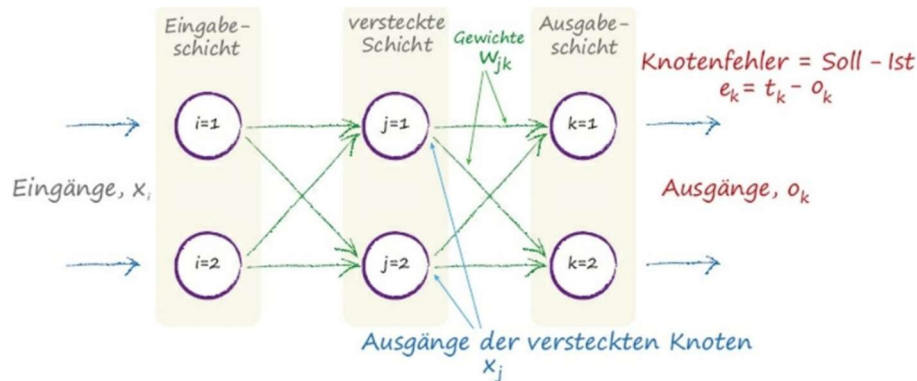


Abbildung 10: Gewichte zwischen der *j* und *i* Schicht

3.3 Das Gradientenverfahren

Bei dem *Gradientenverfahren* wird, vereinfacht ausgedrückt, der Fehler $[e]$ durch Anpassung der Gewichtswerte minimiert. Es wird versucht, den Fehler $[e]$ gegen 0 laufen zu lassen (s. Abb. 11). Dabei ist es wichtig, nicht in einem lokalen Minimum zu landen sondern das optimale Minimum ausfindig zu machen.⁵

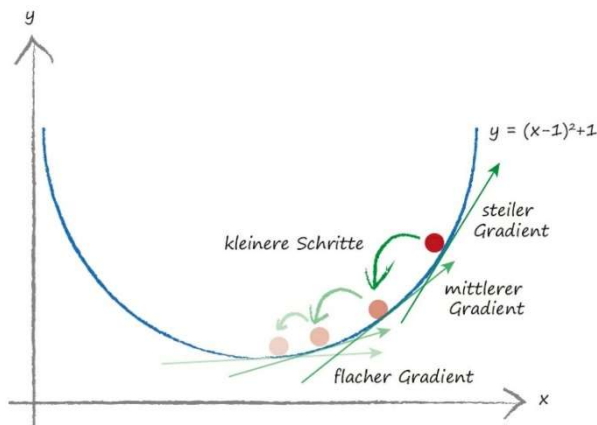


Abbildung 11: Das *Gradientenverfahren*

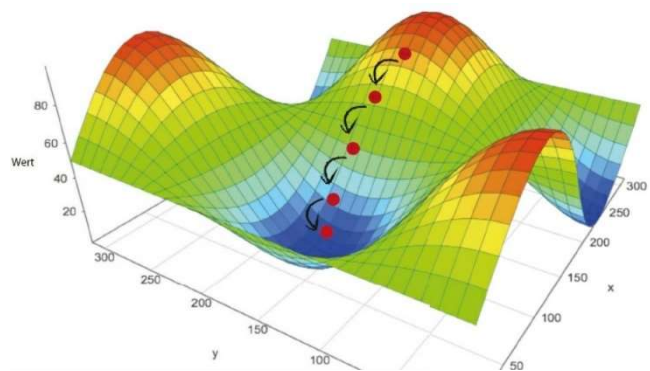


Abbildung 12: Das *Gradientenverfahren* bei 3-Dimensionen

Dafür wird eine weitere Formel genutzt, die für den *n-dimensionalen* Raum gilt. In Abbildung 12 ist eine mögliche Fehlerfunktion für den drei-dimensionalen Raum dargestellt. Die Formel⁶, welche die Ableitung (Gradient) des Fehlers $[e]$ in Abhängigkeit der *weights* W_{jk} angibt, lautet:

⁵ Vgl: Rashid (2017). Neuronale Netze selbst programmieren, S.78

⁶ Vgl: Rashid (2017). Neuronale Netze selbst programmieren, S.85

$$\frac{\partial E}{\partial w_{jk}} = -(t - o_k) * \text{sigmoid}\left(\sum_j w_{jk} * o_j\right) (1 - \text{sigmoid}\left(\sum_j w_{jk} * o_j\right)) * o_j$$

Durch Substitution der einzelnen Teillieder der Funktion wird diese umgeformt und zur Anpassung der Gewichte genutzt:

$$\Delta w_{jk} = E_k * o_k (1 - o_k) * o_j^T$$

Die Gewichte werden nicht direkt zur Anpassung der Gewichte genutzt, sondern durch eine *Learning rate* [α] moderiert. So erhält man folgende Gleichung⁷:

$$\text{new } w_{jk} = \text{old } w_{jk} - \alpha * \Delta w_{jk}$$

4 Handschrifterkennung: Zahlen

4.1 Benötigte Hardware und Software

Für den nachfolgenden Code wurde *Python* verwendet; die verwendete Version entspricht 3.10. Als IDE wurde die *PyCharm Community Edition*⁸ genutzt. Insgesamt wurden 4 Bibliotheken eingesetzt, darunter *Numpy*⁹ für einfache Matrizenrechnung, *Scipy*¹⁰ für die *Sigmoid* Funktion, *Matplotlib*¹¹ um die Ergebnisse grafisch darzustellen, und *Tkinter*¹² um ein passendes GUI zu kreieren. Es wurde bewusst auf Bibliotheken wie *Pytorch* oder *Tensorflow* verzichtet, da diese das Programm zu simpel gestaltet hätten.

⁷ Vgl: Rashid (2017). Neuronale Netze selbst programmieren, S.86

⁸ JetBrains, o. D.

⁹ NumPy, 2023

¹⁰ SciPy, 2024

¹¹ Matplotlib, 2012

¹² Tkinter — Python Interface To TCL/TK, o. D.

4.2 Aufbau



Abbildung 13: Ausschnitt der *mnist_train* Excel Tabelle

gewünschte Ausgabewert des Netzes. Die Zahlen werden durch den Befehl „*split*“ bei jedem Komma separiert. Jede der Zahlen repräsentiert einen Pixel mit einem bestimmten Grauwert.

Das Programm, (s. Abb. 14), wandelt die Pixelwerte in ein Feld um, welches stets 28 x 28 Pixel misst. Die Zahl lässt sich in Abbildung 15 feststellen:

```
import numpy as np
import matplotlib.pyplot as plt

#open file "r" for only read
data_file = open("mnist_train.csv", "r")
data_list = data_file.readlines()
data_file.close()

#split data strings at "," and put into array
all_values = data_list[22].split(",")
image_array = np.asfarray(all_values[1:]).reshape((28, 28))
#scale data for sigmoid function
scale_inputs = (np.asfarray(all_values[1:]) / 255.0 * 0.99) + 0.01
print(scale_inputs)
#invert black white colors
inverted_image_array = 1 - image_array

# Plote das invertierte Bild
plt.imshow(inverted_image_array, cmap="gray", interpolation="None")
plt.show()
```

Abbildung 14: Öffnen der *mnist_train* Daten

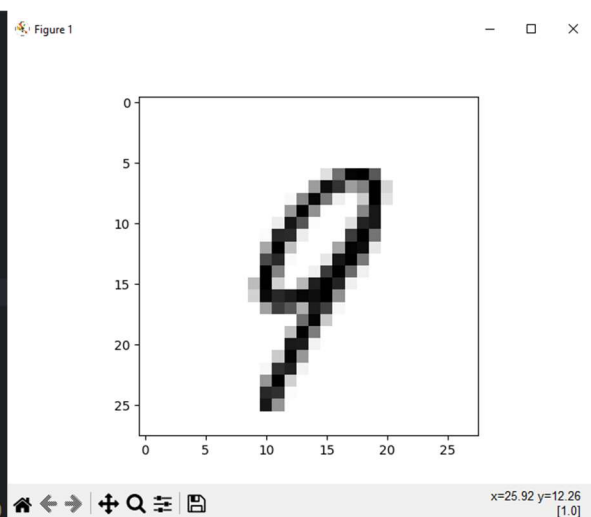


Abbildung 15: Zahl aus der Excel Tabelle

¹³ Dato-on, 2018

5 Code-Implementierung

5.1 Grundstruktur des Codes

```
import numpy as np
import scipy

class neuralNetwork:

    def __init__(self):
        pass

    def train(self):
        pass

    def query(self):
        pass
```

Für das neuronale Netz werden die abgebildeten Bibliotheken importiert, es wird eine *class* mit dem Namen *neuralNetwork* erstellt. Innerhalb dieser *class* werden die Funktionen des Netzes definiert, (s. Abb. 16).

Abbildung 16: *class neuralNetwork*

Die erste Funktion erstellt und definiert die Werte, mit denen das neuronale Netz arbeitet. Das bedeutet, dass dort unter anderem die Werte der *weights* zufällig generiert werden. Außerdem wird die Aktivierungsfunktion festgelegt (s. Abb. 17).

```
def __init__(self, input_nodes, hidden_nodes, output_nodes, learning_rate):
    self.inodes = input_nodes
    self.hnodes = hidden_nodes
    self.onodes = output_nodes

    #define weights
    self.wih = np.random.normal(loc: 0.0, pow(self.hnodes, -0.5), size: (self.hnodes, self.inodes))
    self.who = np.random.normal(loc: 0.0, pow(self.onodes, -0.5), size: (self.onodes, self.hnodes))

    #define learning rete
    self.lr = learning_rate

    #sigmoid function
    self.activation_function = lambda x: scipy.special.expit(x)

    pass
```

Abbildung 17: Erste Funktion

```
def query(self, input_list):
    inputs = np.array(input_list).reshape(-1, 1)

    #into hidden layer
    hidden_inputs = np.dot(self.wih, inputs)
    #from hidden layer
    hidden_outputs = self.activation_function(hidden_inputs)

    #into final layer
    final_inputs = np.dot(self.who, hidden_outputs)
    #from final layer
    final_outputs = self.activation_function(final_inputs)

    #output
    return final_outputs
```

Abbildung 18: query Funktion

Die *query* Funktion dient dazu, das neuronale Netz zu evaluieren, bzw. es abzufragen. Darüber hinaus berechnet diese auch die *outputs* (s. Abb. 18).

Die *train* Funktion sorgt dafür, dass das neuronale Netz einen Trainingsschritt durchläuft in dem Gewichte angepasst werden. Durch die *inputs* und *targets* berechnet diese Funktion den Fehler [*e*] und passt diesen autonom an, Abbildung 19.

```
def train(self, input_list, targets_list):
    inputs = np.array(input_list).reshape(-1, 1)
    targets = np.array(targets_list).reshape(-1, 1)

    #into and form hidden layer
    hidden_inputs = np.dot(self.wih, inputs)
    hidden_outputs = self.activation_function(hidden_inputs)
    #into and form final layer
    final_inputs = np.dot(self.who, hidden_outputs)
    final_outputs = self.activation_function(final_inputs)
    #get error
    output_errors = targets - final_outputs
    hidden_errors = np.dot(self.who.T, output_errors)

    #weight correction
    self.who += self.lr * np.dot((output_errors * final_outputs * (1.0 - final_outputs)),
                                np.transpose(hidden_outputs))
    self.wih += self.lr * np.dot((hidden_errors * hidden_outputs * (1.0 - hidden_outputs)),
                                np.transpose(inputs))
    pass
```

Abbildung 19: train Funktion

5.2 Trainingsprozess

Um das Training effizienter zu gestalten, wird der repetitive Charakter des menschlichen Lernprozesses imitiert indem mehrfach über alle Trainingsdaten gegangen wird. Die Anzahl der Wiederholungen werden *epochs* genannt. Wie bereits zuvor wird das *label* ausgeschlossen, die Werte werden dafür angepasst (s. Abb. 20).

```
for epoch in range(epochs):
    for record in trainigs_data_list:
        all_values = record.split(",")
        inputs = (np.asfarray(all_values[1:]) / 255 * 0.99) + 0.01
        targets = np.zeros(output_nodes) + 0.01
        targets[int(all_values[0])] = 0.99
        n.train(inputs, targets)
        pass
    print("training passed")
    pass
```

Abbildung 20: Code für epoch

```
scorecard = []

test_data_file = open("mnist_test.csv", "r")
test_data_list = test_data_file.readlines()
test_data_file.close()

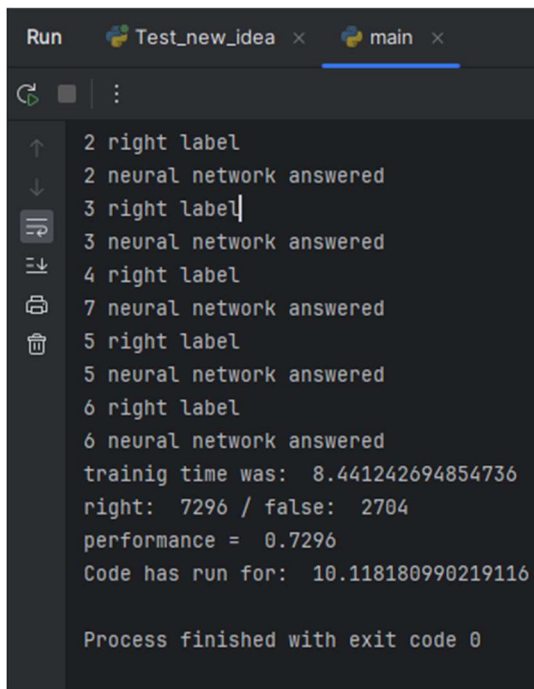
for record in test_data_list:
    all_values = record.split(",")
    correct_label = int(all_values[0])
    print(correct_label, "right label")

    inputs = (np.asfarray(all_values[1:]) / 255 * 0.99) + 0.01
    outputs = n.quary(inputs)
    label = np.argmax(outputs)
    print(label, "neural network answered")

    if(label == correct_label):
        scorecard.append(1)
        right_answer = right_answer + 1
    else:
        scorecard.append(0)
        false_answer = false_answer + 1
    pass
pass
```

Abbildung 21: Öffnen der mnist test daten

Anschließend kann der Trainingserfolg des Netzes durch Tests gemessen werden. Davor wird eine Liste mit dem Titel *scoreboard* erstellt, in die eine *1* für jede richtige Antwort, für jede falsche Antwort eine *0* eingefügt wird. Hiermit wird die Leistung des Netzes erhoben (s. Abb. 21).



```
Run Test_new_idea x main x
2 right label
2 neural network answered
3 right label
3 neural network answered
4 right label
7 neural network answered
5 right label
5 neural network answered
6 right label
6 neural network answered
training time was: 8.441242694854736
right: 7296 / false: 2704
performance = 0.7296
Code has run for: 10.118180990219116

Process finished with exit code 0
```

Abbildung 22: Testresultate

Am Ende der Abfrage zeigt Abbildung 22, dass das neuronale Netz effizient gearbeitet hat: es weist eine *performance* von 0,7296 auf, was einer Trefferquote von aufgerundet 73% entspricht.

6 Evaluation

Die Leistungsfähigkeit des trainierten neuronalen Netzes hängt von drei Faktoren ab, den *hidden nodes*, der *learning rate* und den *epochs*. Bei der Auswertung soll herausgefunden werden, inwiefern diese Faktoren die Leistung beeinflussen und welche Parameter die besten Ergebnisse erzielen.

6.1 Gängige Parameter

Bei neuronalen Netzen ist es nicht ungewöhnlich, Parameter aus Internetforen zu nutzen. Das Problem ist, dass unterschiedliche Foren andere Parameter als die jeweils Besten bestimmen. Allgemein ist festzustellen, dass sich viele Forscher mit einem Wert von 0.1 für die *learning rate* zufriedengeben.¹⁴ Für die *epochs* sieht es etwas anders aus: Hier sind die Werten 6^{15} und 11^{16} am meisten vertreten. Ein anderer Ansatz ist die Anzahl der *hidden layers* mal 3.¹⁷ Das Hauptproblem sind die *hidden nodes*, welche je nach Größe und Ausmaß des Netzes angepasst werden müssen. Im Hinblick auf die kommende Formel ist es wichtig zu erwähnen, dass N für die Anzahl steht,

¹⁴ The Learning Rate, o. D.

¹⁵ manmayi, 2023

¹⁶ DataScientest, 2023

¹⁷ How Many Epochs Should I Train My Model With?, 2019

darüber hinaus steht h für *hidden nodes*, s für die Menge der Trainingsdaten, i für die *input nodes* und o für die *output nodes*. Es gibt mehrere Formeln für die *hidden nodes*:

$$N_h = \frac{N_s}{(\alpha * (N_i + N_o))}$$

Bei dieser Formel¹⁸ gilt zu beachten, dass der Wert für $[\alpha]$ nicht zu groß werden darf, da sonst die Anzahl der *hidden nodes* einen zu geringen Wert annehmen. Weshalb für $[\alpha]$ folgende Definitionsmenge gilt:

$$D_\alpha = \{\alpha \in R | 0 < \alpha < 6\}$$

Um diese Formel zu testen, werden für $[\alpha]$ die Werte 1, 2 und 5 verwendet, womit für $N_{h_{1;2;3}}$ die gerundeten Ergebnisse 75, 28 und 15 vorliegen. Eine andere Formel zur Berechnung der *hidden nodes* sieht folgendermaßen aus:

$$N_h = \sqrt{N_i * N_o}$$

Dieser Term¹⁹ erweckt den Anschein, als würden keine großen Nachteile entstehen. Somit gilt gerundet für N_{h_4} das Ergebnis 88. Die nächste Formel²⁰ gibt eine weitaus höhere Anzahl für die *hidden nodes* zurück.

$$N_h = \frac{2}{3} * N_i$$

Das gerundete Ergebnis für N_{h_5} ist 523. Anhand der Abbildung 23 lässt sich klar erkennen, dass das Netz mit 11 *epochs* und 523 *hidden nodes* am besten, mit einer *accuracy* von 0.9735, abschneidet. Durch die Grafiken wird klar, dass mehrere *epochs* und mehr *hidden nodes* ein besseres Ergebnis erzielen.

¹⁸ How To Choose The Number Of Hidden Layers And Nodes in A Feedforward Neural Network?, 2010

¹⁹ Masters, T. (1993). *Practical neural network recipes in C++*. Morgan Kaufmann.

²⁰ TRAINING AN ARTIFICIAL NEURAL NETWORK, o. D.

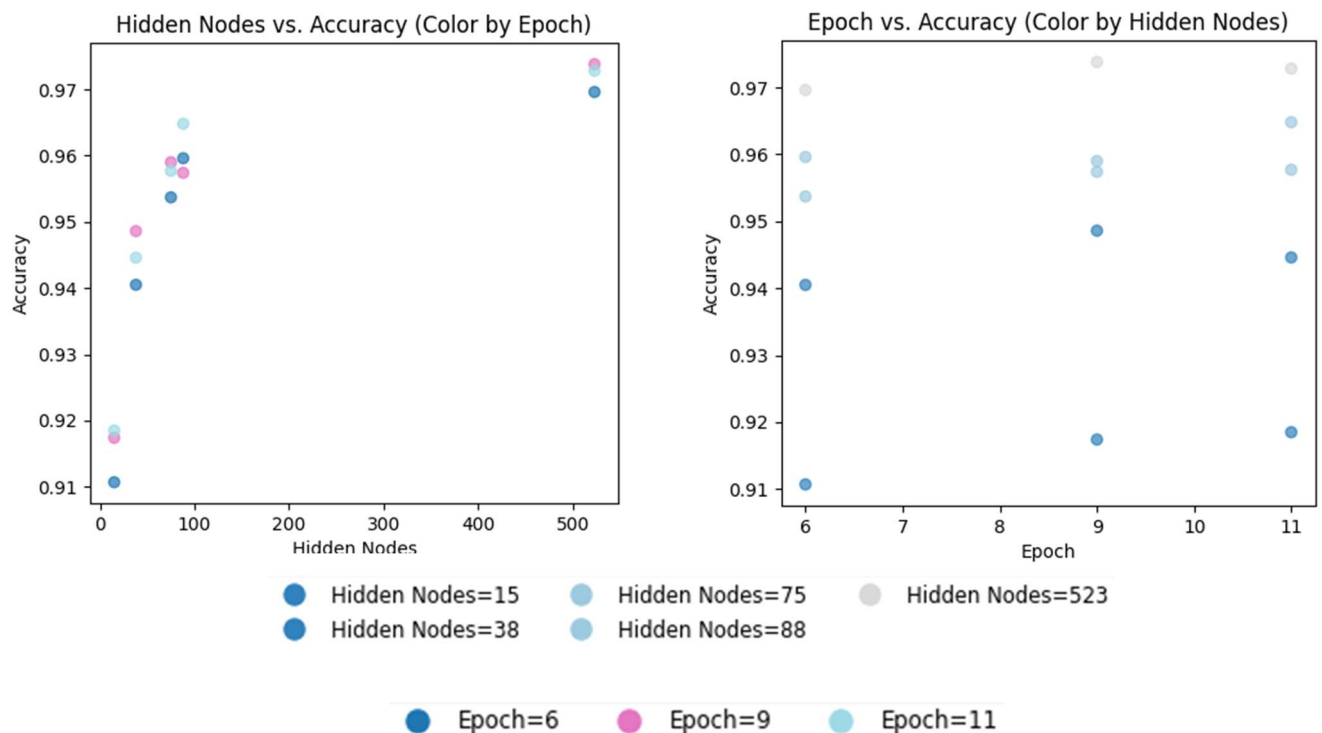


Abbildung 23: Grafik Small Dataset

6.2 Unterschiedliche Parameter

Die Werte für die *epochs* die *learning rate* und die *hidden nodes* können auch noch andere Werte annehmen. Deshalb ist das Ziel, unterschiedliche Parameter mit einer größeren Differenz auszuprobieren. Um dieses Ziel zu erreichen, werden folglich Listen in den Code integriert, (s. Abb. 24) mit welchem die Parameter festgelegt werden können.

```
hidden_nodes_values = np.array([10, 100, 200, 300, 500, 784])
learning_rate_values = np.array([0.01, 0.1, 0.3, 0.5, 0.7, 1.0])
epoch_values = np.array([1, 3, 5, 10, 15])
```

Abbildung 24: Parameter arrays

Jede Kombination aus den Listen wird berechnet. Daraus entstehen die Parameter, um die Kombinationen besser zu repräsentieren. In der *for* Schleife wird jede Kombination trainiert und getestet, dabei wird jede *accuracy* mit den zugehörigen Parametern in einem *array* gespeichert, (s. Abb. 25).

```
# Trainierte Genauigkeiten und zugehörige Parameter speichern
data = []

# Gruppieren der Parameter nach Lernrate und Epochen
grouped_parameters = {}
for learning_rate in learning_rate_values:
    for epoch in epoch_values:
        grouped_parameters[(learning_rate, epoch)] = []

# Trainiertes neuronales Netzwerk simulieren und Daten sammeln
for hidden_nodes in hidden_nodes_values:
    sh_time = time.time()
    print("hn: ", hidden_nodes)
    for learning_rate in learning_rate_values:
        sl_time = time.time()
        print("lr: ", learning_rate)
        for epoch in epoch_values:
            se_time = time.time()
            print("ep: ", epoch)
            accuracy = trained_neural_network(hidden_nodes, learning_rate, epoch)
            data.append((hidden_nodes, learning_rate, epoch, accuracy))
            grouped_parameters[(learning_rate, epoch)].append((hidden_nodes, accuracy))
```

Abbildung 25: Gruppieren der Daten

Mithilfe der Bibliothek *Matplotlib* werden alle diese Wert graphisch dargestellt, (s. Abb 26 und 27)

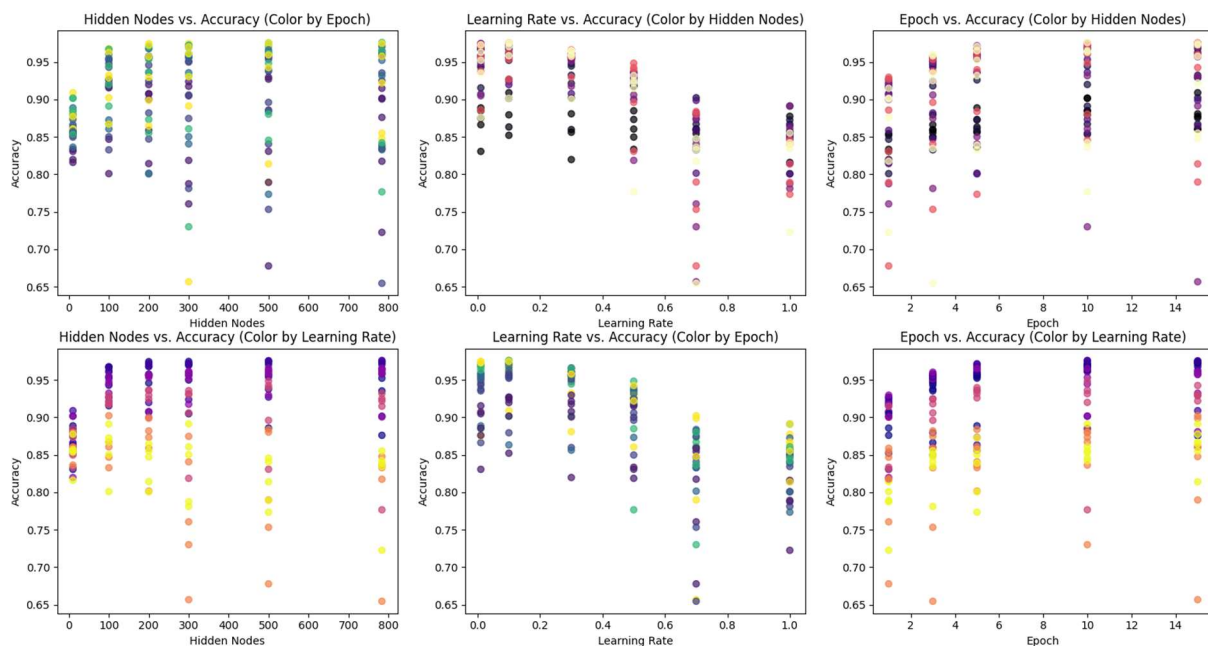


Abbildung 26: Grafik Big Dataset

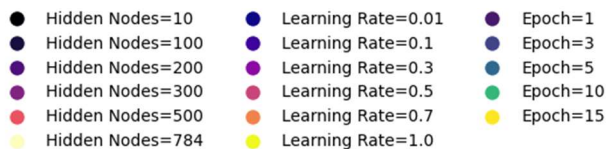


Abbildung 27: Farben Big Dataset

Aus dieser Grafik lassen sich mehrere Tendenzen ablesen und einige Vermutungen bestätigen.

Bei „*hidden nodes vs accuracy*“ ist ein Trend zu erkennen, der die Vermutung „mehr *hidden nodes* = besseres Ergebnis“, teilweise bestätigt.

Es ist eine deutliche Zunahme der *accuracy* zu erkennen, die jedoch im Laufe des Graphen abnimmt. Durch die farbliche Markierung ist zu erkennen, dass die dritte aufgestellte Formel zur Berechnung der *hidden nodes*, ein guter Ansatz war und somit der optimale Wert für die *hidden nodes* um 500 liegt. Ebenfalls durch die Farben zu erkennen ist die *learning rate* in Korrelation zu den *hidden nodes*, woraus folgt, dass eine höhere *learning rate* die *accuracy* negativ beeinflusst, was auch durch die Graphen „*learning rate vs accuracy*“ bestätigt wird, was bedeutet das eine *learning rate* von ca. 0.1 als optimal angesehen werden kann.

Aus der Grafik „*epoch vs accuracy*“ geht hervor, dass eine Anzahl von mehr als 10 *epochs* keinen großen Einfluss auf die *accuracy* nimmt, was bedeutet, dass der optimale Wert für die *epochs* für die verwendeten Trainingsdaten im Bereich von 10 liegt. Am besten haben folgende Kombinationen abgeschnitten:

(500, 0.1, 10, 0.9753)

(500, 0.1, 15, 0.9755)

(784, 0.1, 10, 0.976)

Dieser Befund bekräftigt die Vermutung des optimalen Werts für die *hidden nodes* die *epochs* und der *learning rate*.

6.3 Generalization und Orkham's razor

Rückblickend auf die Ergebnisse fällt die dritte Kombination aus dem Rahmen. 784 *hidden nodes* entsprechen exakt der Anzahl der *input nodes* was die Frage aufwirft, warum nicht immer die *hidden nodes* den *input nodes* gleichzusetzen sind, da diese Kombination das beste Resultat erbracht hat.

Bei einem neuronalen Netz geht es darum, ausgewählte Daten anhand ihrer Charakteristiken zu analysieren, um somit neue und unbekannte Daten zu erfassen und zu klassifizieren, wobei oft Daten zusammengefasst werden. Dieser Prozess wird auch *Generalization* genannt, was bedeutet das ein trainiertes neuronales Netz genauso gut in Testsituationen als auch in neuen Situationen funktioniert. Das Problem ist, dass ein neuronales Netz, bei dem die *hidden nodes* den *input nodes* entsprechen, nicht dieses Verfahren anwendet, da es keine Klassifizierung vornimmt oder Daten zusammenfasst.

Um eine gute *Generalization* zu erzielen ist es essenziell, ein so simpel wie möglich gehaltenes Netz zu trainieren, was trotz seiner Größe die Daten vollständig erfasst. Zu diesem Zweck kann ein Prinzip namens *Orkham's razor* angewendet werden. Dieses Prinzip besagt: Je komplexer ein

System, desto anfälliger ist es für Störungen und Fehler. Was für das neuronale Netz so viel bedeutet wie: Je mehr *weights* und *hidden inputs*, umso anfälliger für Fehlinterpretationen. Folglich ist das Resultat (784, 0.1, 10, 0.976) mit Vorsicht zu betrachten.

7 Demonstration

Vorführen des Live-Testing Codes.

8 Fazit

Im Rahmen dieser Facharbeit wurde zunächst die generelle Funktionsweise von künstlichen Intelligenzen erläutert. Daraufhin wurde eine selbst erstellte KI zur Erkennung von Ziffern vorgeführt und spezifisch betrachtet. Die KI wurde mit einer Varianz an Parametern idealisiert. Der Fokus wurde auf die mathematische Funktionsweise und die Evaluation der *performance* gelegt. Mithilfe des Vortrags soll das Interesse der Teilnehmenden des Kurses an neuronalen Netzen in Bezug auf den Überbegriff KI gefestigt, da diese in naher Zukunft, beispielsweise in Form des Hilfsmittels ChatGPT, an Signifikanz gewinnen werden.

9 Literaturverzeichnis

9.1 Buchquellen

Rashid, T. (2017): Neuronale Netze selbst programmieren (F. Langenau, Übers.) O'Reilly.

Hagan, M., Demuth, H. & Beale, M. (2014). Neural network design (2. Aufl.) [PDF]. <https://hagan.okstate.edu/NNDesign.pdf>

Masters, T. (1993). Practical neural network recipes in C++. Morgan Kaufmann.

9.2 Internetquellen

Spiegel. (2024, 15. Februar). Empfehlungen für KI-Nutzung an Schulen. Abgerufen am 15. Januar 2024, von <https://www.spiegel.de/panorama/bildung/chatgpt-und-co-experten-geben-empfehlungen-fuer-ki-nutzung-an-schulen-a-4e58f272-315c-4ce3-86e7-9f9e4ee17bd1>

Tiedemann, M. (2018, 29. Mai). KI, Künstliche Neuronale Netze, Machine Learning, Deep Learning. Abgerufen am 15. Februar 2024, von https://www.alexanderthamm.com/de/blog/ki_artificial-intelligence-ai-kuenstliche-neuronale-netze-machine-learning-deep-learning/

jetbrains. (o. D.). PyCharm the Python IDE for professional developers. jetbrains.com. Abgerufen am 18. Februar 2024,

von <https://www.jetbrains.com/pycharm/>

Matplotlib. (2012). Choosing Colormaps in Matplotlib. [matplotlib.org](https://matplotlib.org/stable/users/explain/colors/colormaps.html). Abgerufen am 18. Februar 2024, von <https://matplotlib.org/stable/users/explain/colors/colormaps.html>

NumPy. (2023, 16. September). Abgerufen am 18. Februar 2024, von <https://numpy.org>

SciPy. (2024, 20. Januar). Abgerufen am 18. Februar 2024, von <https://scipy.org>

Tkinter — Python Interface to TCL/TK. (o. D.). Abgerufen am 18. Februar 2024, von <https://docs.python.org/3/library/tkinter.html>

Dato-on, D. (2018). MNIST in CSV. [kaggle.com](https://www.kaggle.com/datasets/oddrational/mnist-in-csv). Abgerufen am 18. Februar 2024, von <https://www.kaggle.com/datasets/oddrational/mnist-in-csv>?r=

How many epochs should I train my model with? (2019). Abgerufen am 18. Februar 2024, von <https://gretel.ai/gretel-synthetics-faqs/how-many-epochs-should-i-train-my-model>
with#:~:text=The%20right%20number%20of%20epochs,again%20with%20a%20higher%20value.

manmayi. (2023, 28. Februar). Choose optimal number of epochs to train a neural network in Keras. [geeksforgeeks.org](https://www.geeksforgeeks.org/choose-optimal-number-of-epochs-to-train-a-neural-network-in-keras/). Abgerufen am 18. Februar 2024, von <https://www.geeksforgeeks.org/choose-optimal-number-of-epochs-to-train-a-neural-network-in-keras/>

DataScientest. (2023, 2. Juni). Epoch: an essential notion in real-time programming. [datascientest.com](https://datascientest.com/en/epoch-an-essential-notion#:~:text=The%20number%20of%20epochs%20is%20a%20hyperparameter%20that%20must%20be,iterative%20process%20of%20gradient%20descent). Abgerufen am 18. Februar 2024, von <https://datascientest.com/en/epoch-an-essential-notion#:~:text=The%20number%20of%20epochs%20is%20a%20hyperparameter%20that%20must%20be,iterative%20process%20of%20gradient%20descent>.

The learning rate. (o. D.). [andreaperlato.com](https://www.andreaperlato.com/theorypost/the-learning-rate/#:~:text=The%20range%20of%20values%20to,starting%20point%20on%20your%20problem). Abgerufen am 18. Februar 2024, von <https://www.andreaperlato.com/theorypost/the-learning-rate/#:~:text=The%20range%20of%20values%20to,starting%20point%20on%20your%20problem>.

How to choose the number of hidden layers and nodes in a feedforward neural network? (201 n. Chr., Juli). [stackexchange.com](https://stats.stackexchange.com/questions/181/how-to-choose-the-number-of-hidden-layers-and-nodes-in-a-feedforward-neural-netw). Abgerufen am 18. Februar 2024, von <https://stats.stackexchange.com/questions/181/how-to-choose-the-number-of-hidden-layers-and-nodes-in-a-feedforward-neural-netw>

TRAINING AN ARTIFICIAL NEURAL NETWORK. (o. D.). [solver.com](https://www.solver.com/training-artificial-neural-network-intro). Abgerufen am 18. Februar 2024, von <https://www.solver.com/training-artificial-neural-network-intro>

10 Abbildungsverzeichnis

Abbildung 1: Skizze von Neuronen in einem Taubengehirn (S.4)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.31)

Abbildung 2: Schema einer einfachen Vorhersagemaschine (S.4)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.3)

Abbildung 3: Alternative Beschreibung der Vorhersagemaschine (S.4)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.3)

Abbildung 4: Einfaches neuronales Netz (S.4)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.39)

Abbildung 5: Erweitertes Modell eines künstlichen Neurons (S.5)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.42)

Abbildung 6: Bildung der inputs für die zweite Schicht (S.5)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.49)

Abbildung 7: Matrixmultiplikation (S.5)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.48)

Abbildung 8: Sigmoid funktion (S.6)

Quelle: eigene Darstellung

Abbildung 9: Problem bei mehreren inputs (S.6)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.59)

Abbildung 10: Gewichte zwischen der j und i Schicht (S.7)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.82)

Abbildung 11: Das Gradientenverfahren (S.7)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.76)

Abbildung 12: Gradientenverfahren bei 3-Dimensionen (S.7)

Quelle: Rashid, T. (2017): Neuronale Netze selbst programmieren (S.77)

Abbildung 13: Ausschnitt der mnist train Excel Tabelle (S.9)

Quelle: eigene Darstellung

Abbildung 14: Öffnen der mnist train daten (S.9)

Quelle: eigene Darstellung

Abbildung 15: Zahl aus der Excel Tabelle (S.9)

Quelle: eigene Darstellung

Abbildung 16: class neuralNetwork (S.10)

Quelle: eigene Darstellung

Abbildung 17: Erste Funktion (S.10)

Quelle: eigene Darstellung

Abbildung 18: quarry Funktion (S.11)

Quelle: eigene Darstellung

Abbildung 19: train Funktion (S.11)

Quelle: eigene Darstellung

Abbildung 20: Code für epoch (S.12)

Quelle: eigene Darstellung

Abbildung 21: Öffnen der mnist test daten (S.12)

Quelle: eigene Darstellung

Abbildung 22: Testresultate (S.13)

Quelle: eigene Darstellung

Abbildung 23: Grafik Small Dataset (S.15)

Quelle: eigene Darstellung

Abbildung 24: Parameter arrays (S.15)

Quelle: eigene Darstellung

Abbildung 25: Gruppieren der Daten (S.16)

Quelle: eigene Darstellung

Abbildung 26: Grafik Big Dataset (S.16)

Quelle: eigene Darstellung

Abbildung 27: Farben Big Dataset (S.16)

Quelle: eigene Darstellung